# Solution: Contains Duplicate

## Problem Statement

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

**Example 1:**

```
Input: nums= [1, 2, 3, 4]
Output: false
Explanation: There are no duplicates in the given array.
```

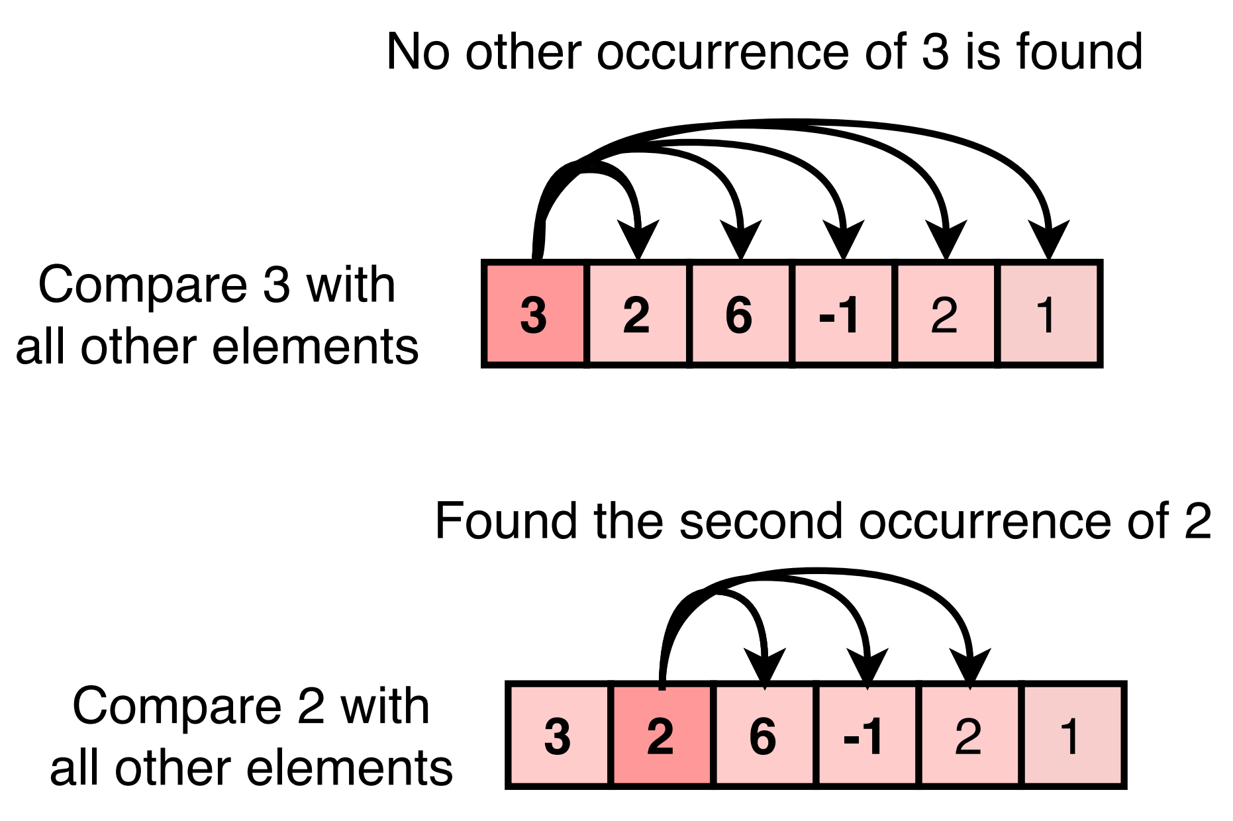
**Example 2:**

```
Input: nums= [1, 2, 3, 1]
Output: true
Explanation: '1' is repeating.
```

## Solution

### Approach 1: Brute Force

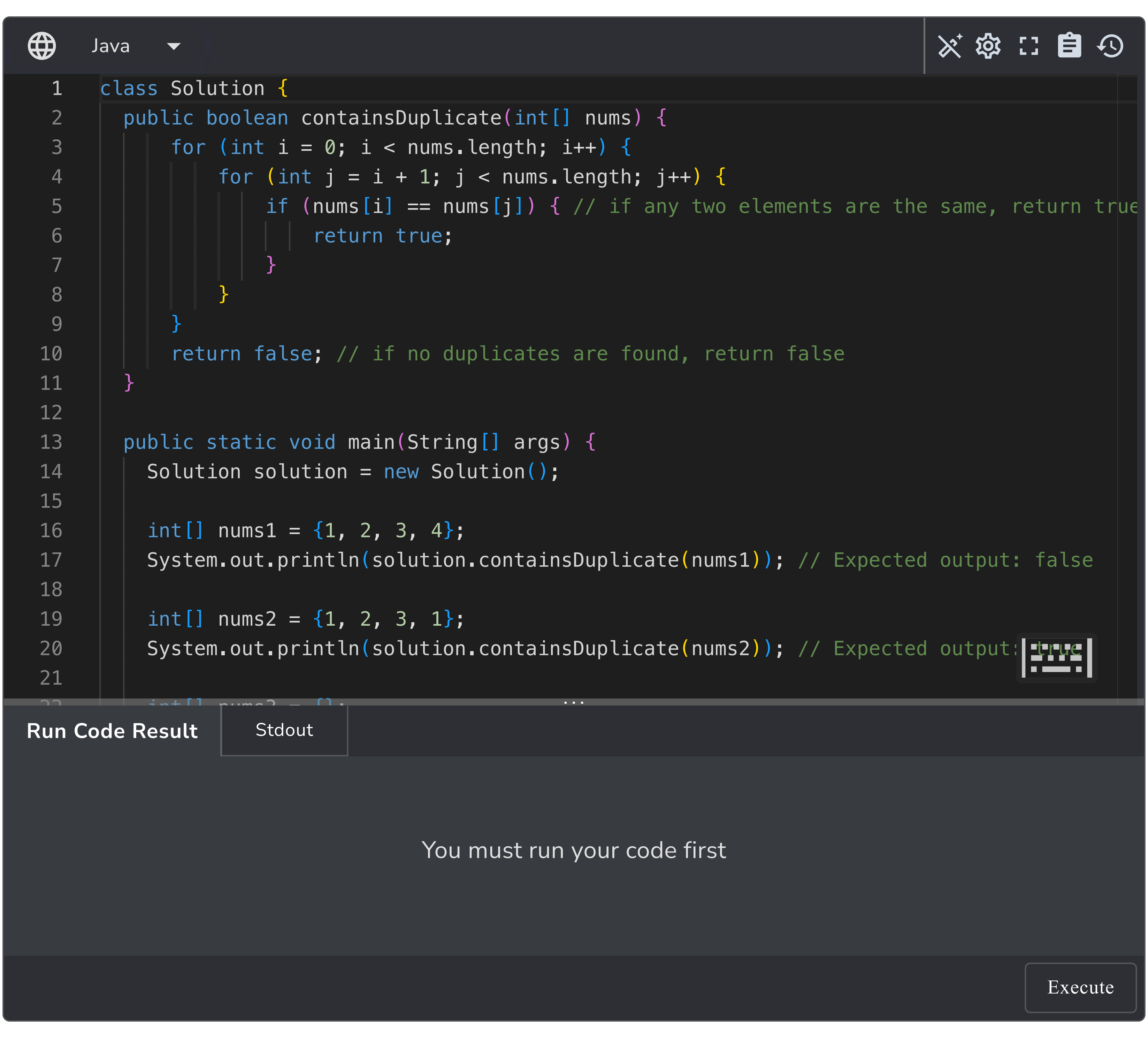
We can use a brute force approach and compare each element with all other elements in the array. If any two elements are the same, we'll return `true`. If we've gone through the entire array and haven't found any duplicates, we'll return `false`.


No other occurrence of 3 is found
Compare 3 with all other elements
3
2
6
-1
2
1
Found the second occurrence of 2
Compare 2 with all other elements
3
2
6
-1
2
1

Brute force approach to solve Contains Duplicate

## Code

Here is the code for this algorithm:



```java
class Solution {
  public boolean containsDuplicate(int[] nums) {
    for (int i = 0; i < nums.length; i++) {
      for (int j = i + 1; j < nums.length; j++) {
        if (nums[i] == nums[j]) { // if any two elements are the same, return true
          return true;
        }
      }
    }
    return false; // if no duplicates are found, return false
  }

  public static void main(String[] args) {
    Solution solution = new Solution();

    int[] nums1 = {1, 2, 3, 4};
    System.out.println(solution.containsDuplicate(nums1)); // Expected output: false

    int[] nums2 = {1, 2, 3, 1};
    System.out.println(solution.containsDuplicate(nums2)); // Expected output: true

```

**Time Complexity** The algorithm takes $O(N^2)$ where $N$ is the number of elements in the input array. This is because we need to compare each element with all other elements, which takes $O(N^2)$ time.

**Space Complexity** The algorithm takes $O(1)$, because we only need a few variables to store the indices, which takes constant space.

### Approach 2: Using Hash Set

We can use the `set` data structure to check for duplicates in an array.

Since a `set` can only hold unique elements, we can check if the elements in the given array are present more than once by adding them to a `set`. This way, we can determine if there are any duplicates in the array.

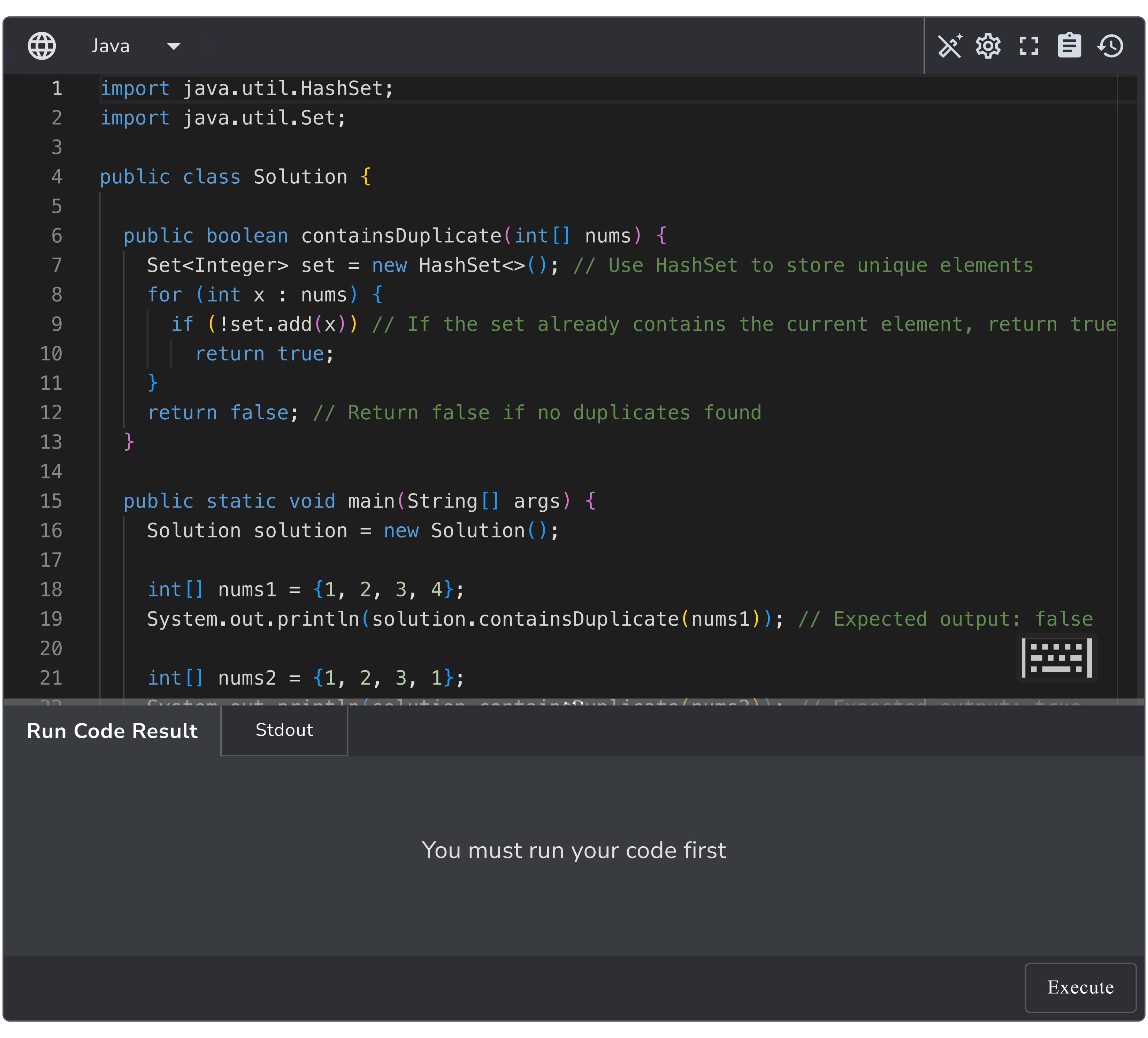
This approach works as follows:

1. A set named `unique_set` is created to store unique elements.
2. The algorithm then iterates through the input array `nums`.
3. For each element "x" in the array, the algorithm checks if "x" is already in the `unique_set`.
    - If "x" is in the `unique_set`, then the algorithm returns True, indicating that a duplicate has been found.
    - If "x" is not in the `unique_set`, then the algorithm adds "x" to the `unique_set`.
4. The iteration continues until all elements in the array have been processed.
5. If no duplicates are found, the algorithm returns False.

This approach utilizes the property of sets to store only unique elements, making it an efficient solution for finding duplicates in an array.

## Code

Here is the code for this algorithm:



```java
import java.util.HashSet;
import java.util.Set;

public class Solution {

  public boolean containsDuplicate(int[] nums) {
    Set<Integer> set = new HashSet<>(); // Use HashSet to store unique elements
    for (int x : nums) {
      if (!set.add(x)) // If the set already contains the current element, return true
        return true;
    }
    return false; // Return false if no duplicates found
  }

  public static void main(String[] args) {
    Solution solution = new Solution();

    int[] nums1 = {1, 2, 3, 4};
    System.out.println(solution.containsDuplicate(nums1)); // Expected output: false

    int[] nums2 = {1, 2, 3, 1};
```

**Time Complexity** The algorithm takes $O(N)$ where $N$ is the number of elements in the input array. This is because we iterate the array only once.

**Space Complexity** The algorithm takes $O(N)$, as it stores the numbers in a set.

### Approach 3: Sorting

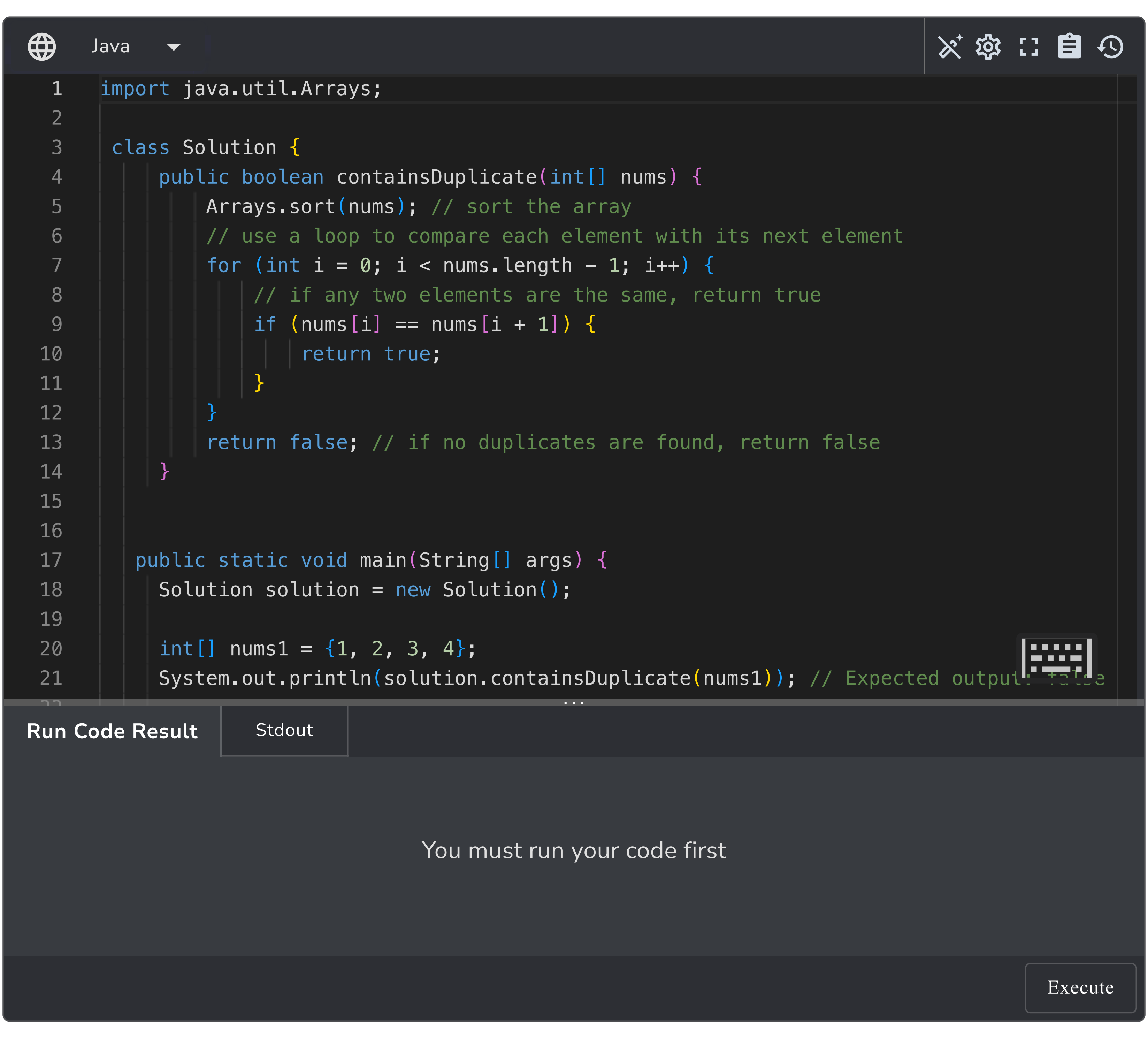
Another approach is to sort the array first and then check for duplicates.

We'll sort the array and then iterate through it, comparing each element with the next one.

If any two elements are the same, we'll return `true`. If we've gone through the entire array and haven't found any duplicates, we'll return `false`.

## Code

Here is the code for this algorithm:



```java
import java.util.Arrays;

 class Solution {
     public boolean containsDuplicate(int[] nums) {
         Arrays.sort(nums); // sort the array
         // use a loop to compare each element with its next element
         for (int i = 0; i < nums.length - 1; i++) {
             // if any two elements are the same, return true
             if (nums[i] == nums[i + 1]) {
                 return true;
             }
         }
         return false; // if no duplicates are found, return false
     }


   public static void main(String[] args) {
     Solution solution = new Solution();

     int[] nums1 = {1, 2, 3, 4};
     System.out.println(solution.containsDuplicate(nums1)); // Expected output: false
```

**Time Complexity**: $O(N * logN)$, where $N$ is the number of elements in the array `nums`. This is because sorting the array takes $O(N * logN)$ time.

**Space Complexity:** $O(1)$ or $O(n)$, depending on the sorting algorithm used. If an in-place sorting algorithm is used, the space complexity will be $O(1)$. If a sorting algorithm that creates a new array is used, the space complexity will be $O(N)$, where $N$ is the number of elements in the array `nums`.

---

← Previous
Problem 2: Contains Duplicate (Easy)

Next →
Problem 3: Left And Right Sum Differences (Easy)

☐ Mark as Completed